

PRÁCTICA 6

COMUNICACIONES: WI-FI, MQTT

Juan Pedro Bandera Rubio (jpbandera@uma.es)

Tu ESP32-S3 puede comunicarse tanto vía Wi-Fi como Bluetooth. Esto es algo tremendamente importante para tu microbot, que gracias a esta conectividad adquiere las siguientes capacidades, entre otras:

- **Puede ser teleoperado:** Esto no nos gusta especialmente en esta asignatura, donde queremos robots autónomos, pero tener esta capacidad de todos modos siempre es muy útil.
- **Puede trabajar en equipo:** Comunicarse con otros robots le permite coordinar las acciones a nivel de flota (piensa, por ejemplo, en 100 robots trabajando en un almacén, un escenario en absoluto extraño hoy día). Comunicarse con las personas que trabajan con él le permite cooperar con ellas de forma más efectiva y segura.
- **Se vuelve muchísimo más listo:** En el ESP32-S3 no van a caber componentes de navegación social avanzada, reconocimiento de objetos y personas basado en LLM, planificadores generativos, etc. No va a ser posible procesar imágenes a 30 fotogramas por segundo para describir escenas a partir de objetos detectados, y no pueden guardarse bases de datos enormes, por ejemplo con perfiles de usuario. Pero todo eso sí puede gestionarse en servidores externos, o en la nube. Al poder conectarse a otros equipos, tu microbot puede enviar un prompt, una imagen, una solicitud de información en definitiva, a internet. Y recibir la respuesta generada para determinar su comportamiento.

Sin olvidar que **el robot debe ser capaz de seguir funcionando cuando se pierde la conexión inalámbrica** (para, al menos, moverse hasta buscar un punto con cobertura Wi-Fi sin chocarse con nada), está claro que es buena idea aprovechar estas ventajas.

Sin embargo, establecer y gestionar una conexión inalámbrica, y usar un protocolo de comunicación para intercambiar información a través de la red, puede resultar bastante enrevesado y no pensamos que sea algo con lo que tengáis que lidiar en esta asignatura. Por eso el código de ejemplo *ESP32_S3_espidf_MQTT* te proporciona dos librerías, y un ejemplo de uso para ellas, que te permitirán incorporar estas capacidades a tu ESP32-S3 de forma sencilla. Estas librerías son:

- *ub_esp32_s3_wifi*: Permite gestionar la conexión Wi-Fi de tu ESP32-S3.
- *ub_esp32_s3_mqtt*: Permite que tu ESP32-S3 se comunique con otros dispositivos mediante el protocolo MQTT.

NOTA: De todos modos el código de las librerías se proporciona en abierto. Nada te impide usarlo como base para crear tus propias variantes. Tampoco le pondremos ninguna pega a que decidas usar conexiones Bluetooth, otros protocolos de comunicación en red diferentes a MQTT, comunicación mediante topics ROS2, etc.

LIBRERÍA PARA LA CONEXIÓN WI-FI (*ub_esp32_s3_wifi*)

Acerca de la librería para la gestión de la Wi-Fi, no hay demasiado que decir: simplemente tener claro que tienes que proporcionar el SSID y la PASSWORD de la Wi-Fi a la que quieres conectarte en el archivo *platformio.ini*, y seguir los pasos del ejemplo para establecer la conexión. También es buena idea desconectarse de la Wi-Fi antes de acabar la ejecución de tu programa.

LIBRERÍA PARA LA COMUNICACIÓN MQTT (*ub_esp32_s3_mqtt*)

Esta librería sí requiere más explicación. En primer lugar, se basa en un componente de ESP32 que no está en el conjunto por defecto de PlatformIO, con lo que **para incluirla es necesario hacer lo siguiente:**

- Crear el archivo *idf_component.yml* en el subdirectorio *src* (donde está el *main.c*, vaya). Este archivo contiene dependencias que son necesarias para tu proyecto y gestiona su descarga automática desde el registro de componentes ESP o desde un repositorio Git.
- Rellenar el archivo *idf_component.yml* con las dependencias que necesitamos para compilar y ejecutar el programa. La principal, *espressif/mqtt*. El archivo quedará así:

```
! idf_component.yml
dependencies:
  protocol_examples_common:
    path: ${IDF_PATH}/examples/common_components/protocol_examples_common
  espressif/mqtt:
    version: "*"
  espressif/esp_wifi_remote:
    version: ">=0.10,<2.0"
    rules:
      - if: target in [esp32p4, esp32h2]
```

Donde la última dependencia, la de *espressif/esp_wifi_remote*, no es realmente necesaria en nuestro caso, ya que nuestro 'target' es un esp32s3.

- *Opcionalmente:* Actualizar el includePath de tu proyecto VSC para que incorpore las rutas de los nuevos includes y no te salten avisos al compilar.

El hecho de que sólo haya que hacer esto para incorporar una librería externa nueva al proyecto dice mucho en favor de ESP-IDF.

El hecho de que teóricamente haya otras formas de hacerlo (también siguiendo instrucciones de ESP-IDF), que dan problemas cuando se intentan, no se adaptan bien a PlatformIO aunque teóricamente deberían, hacen saltar errores de compilación inesperados, etc... dice mucho acerca de las dificultades a las que los programadores se tienen que enfrentar a diario.

Si quieres más información sobre estos temas, sigue este enlace: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-component-manager.html>

Una vez tienes la librería correctamente integrada, es hora de usarla y lo primero es entender el protocolo MQTT.

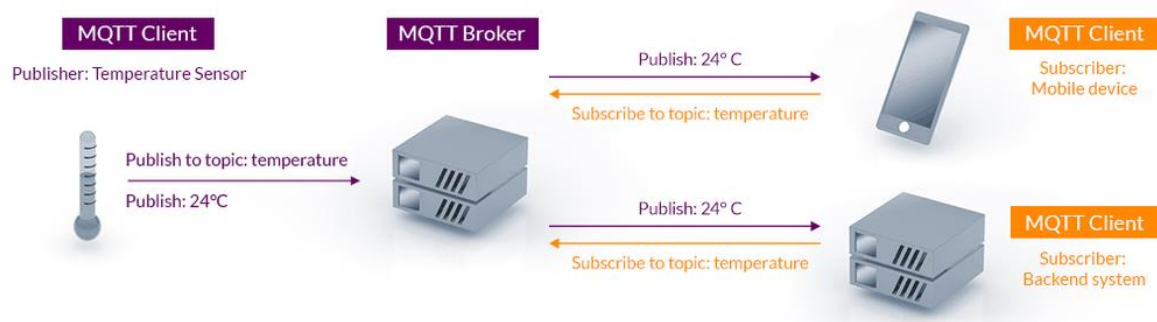
FUNDAMENTOS DE MQTT

Brevemente:

MQTT (<https://mqtt.org/>) es un protocolo de comunicación basado en *publicación/suscripción*, diseñado para ser extremadamente ligero. Es muy empleado en diversos ámbitos, y especialmente en el campo del IoT (Internet of Things).

Básicamente, tienes por una parte el servidor o *broker MQTT*, que es quien se encarga de gestionar los mensajes, que se ordenan en una serie de tópicos (*topics*). Por otro lado, están los clientes MQTT que se encargan de suscribirse a los topics que les interesan, y/o de publicar mensajes en ellos.

MQTT Publish / Subscribe Architecture



Fuente de la imagen: <https://mqtt.org/>

Así, por poner un ejemplo, un programa de monitorización (al que podemos llamar Overlord, por usar un nombre discreto) podría estar suscrito al topic *MQTT environment/presence*, en el que cualquier sensor de presencia de un edificio publicará un mensaje si se detecta a una persona en su rango. También se suscribe a *environment/airquality* en el que los sensores del edificio publican medidas de calidad de aire en diferentes estancias. Finalmente, se suscribe a *robots/status* donde una pequeña flota de robots que patrullan el edificio publican su estado (posición, nivel de batería, tarea actual).

El programa Overlord, considerando la información que se va publicando en los topics a los que se ha suscrito, toma decisiones (por ejemplo, enviar un cierto robot a comprobar una habitación en que se ha detectado a una persona, cuando no debería haber nadie). Esta decisión la comunica escribiendo un mensaje en el topic *robots/update*, al que están suscritos todos los robots.

USO DE MQTT CON LA LIBRERÍA *ub_esp32_s3_mqtt*

La API de esta librería, que implementa un cliente MQTT, es sencilla. Para iniciar el cliente, tienes dos funciones según el broker al que quieras conectarte:

- En el caso de que sea un broker sin cifrado y que use el puerto 1883 (el puerto estándar MQTT para conexiones no cifradas), sólo tienes que indicar la URL del Broker MQTT. Por ejemplo, *mqtt://test.mosquitto.org* es un broker MQTT público para pruebas que pone Mosquitto (<https://test.mosquitto.org/>) a disposición de todo el mundo, y que de hecho puedes usar para probar tus códigos.
- En el caso de que sea un broker con un acceso más controlado, tienes otra función donde puedes indicar la URL del broker, el puerto de conexión empleado, el usuario y la clave. Los profesores te proporcionaremos tus credenciales de acceso para el broker MQTT de Microbótica.

Por lo demás, sigue los pasos del código de ejemplo para suscribirse y publicar. Respecto a esto, puede que te llame la atención el parámetro QoS de las funciones de la librería. En esta asignatura no es necesario que le prestes demasiada atención, pero si quieres profundizar en MQTT, la QoS (Quality of Service) es algo realmente importante para este protocolo. Define, básicamente, la fiabilidad en la entrega de mensajes entre un publicador y un suscriptor. Hay tres niveles, y puedes seleccionar el que quieras cuando te conectas a un topic (bien para recibir mensajes, bien para enviarlos):

- *QoS 0 (como mucho uno)*: Fire-and-forget. Como cliente enviarás el mensaje una vez y te olvidarás de él. Como suscriptor el broker te enviará el mensaje una vez, y ya está.
- *QoS 1 (al menos uno)*: Entrega con acuse de recibo. Cuando se envía un mensaje, si pasa un tiempo y no se recibe acuse de recibo se vuelve a enviar. Tiene el problema de que pueden enviarse mensajes duplicados.
- *QoS 2 (exactamente uno)*: se garantiza que cada mensaje se envía sólo una vez y que se recibe adecuadamente. Es el que más carga el sistema, ya que viene a ser algo así como: envía mensaje -> llega acuse de recibo -> envío mensaje diciendo que el acuse de recibo ha llegado -> llega aceptación de que todo ha ido estupendamente. Son cuatro paquetes de información por mensaje, cuando QoS 0 usa un sólo paquete.

PRÁCTICA EN CLASE: USO DE MENSAJES MQTT CON FORMATO JSON

En un mensaje MQTT puedes publicar texto en el formato que quieras. Sin embargo, una de las opciones más comunes, y recomendable, es usar el formato JSON para estos mensajes.

JSON (<https://www.json.org/json-es.html>) es un formato ligero de intercambio de datos, que surge con la idea de ser fácilmente legible para los humanos y fácilmente manejable para las máquinas. Los datos se suelen organizar en pares *key-value* (clave-valor). Hay miles de páginas web donde dan ejemplos y explicaciones de este formato de datos.

Antes, ESP-IDF integraba la gestión de paquetes JSON, pero ahora, en lugar de eso, proporciona el componente **CJSON** en el registro de componentes, al igual que hace con MQTT. Aquí se explica cómo puedes añadir *espressif/cjson* al archivo *idf_component.yml*: <https://docs.espressif.com/projects/espressif-idf/en/stable/esp32s3/migration-guides/release-6.x/6.0/protocols.html>

Ahora, partiendo del código de ejemplo *ESP32_S3_espidf_MQTT*, del código de ejemplo *ESP32_S3_espidf_ADC_PWM*, y de uno de los códigos de ejemplo que usan temporizadores, tendrás que realizar un nuevo programa que, cada cinco segundos, envíe al topic *ub/test* de nuestro broker MQTT un mensaje JSON con el siguiente formato:

```
{
```

```
"count": <número_del_mensaje>,  
"id": <número_de_tu_grupo>,  
"tags": ["student", "microbotics", "testing"],  
"ADC": <valor_leído_por_ADC>,  
}
```

NOTA: Los valores <valor> representan un número entero que tendrás que rellenar en cada mensaje con su valor correspondiente. Esos valores se ponen tal cual. Como ejemplo, para el mensaje número 4 del grupo número 2, en el que la lectura del ADC vale 230, el mensaje que habría que enviar al topic *ub/test* MQTT es el siguiente:

```
{  
    "count": 4,  
    "id": 2,  
    "tags": ["student", "microbotics", "testing"],  
    "ADC": 230,  
}
```